Automatic Key Modulation Monitoring and Tonicization Detection with Hierarchical Clustering

Dengyun Huang[1] , Yonghua Zhu[1*]
1. Shanghai Film Academy, Shanghai University, Shanghai, China
hdy@shu.edu.cn, zyh@shu.edu.cn
Corresponding Author: Yonghua Zhu Email: zyh@shu.edu.cn

2025 IEEE 8th Advanced Information Technology, Electronic and Automation Control Conference (IAEAC) | 979-8-3315-0951-4/25/$31.00 ©2025 IEEE | DOI: 10.1109/IAEAC65194.2025.11166492

***Abstract*—In a wide range of tasks involving tonal music, key annotation is essential. Among these, the handling of modulation remains a blind spot in most existing approaches. As a result, music analysis often struggles to cope with highly modulating and tonally diverse material. Moreover, with the growing trend of artificial intelligence researches in music generation and style transfer, there is an increasing demand for datasets containing more precise key information. This paper proposes an unsupervised algorithm capable of rapidly and effectively detecting the key of symbolic music, while also addressing any instances of modulation or tonic deviation. We evaluate the method on diverse samples, both with and without modulation, across multiple musical styles. Comparative analysis with established algorithms demonstrates its promising performance.**

Keywords—key detection; key modulation; symbolic music; hierarchical clustering; beam search

I. INTRODUCTION

Key detection has long been a significant topic within Music Information Retrieval (MIR) systems, influencing dataset construction, music analysis, and generative tasks in music research. Traditional key detection methods—excluding those based on deep learning [12, 14]—typically rely on note frequency statistics, note duration statistics, and the comparison of key profile weights using Pearson correlation coefficients [9]. Classical algorithms of this kind are implemented in tools such as music21 [11]. For key modulation detection, the neo-Riemannian framework [10] is frequently employed to infer tendencies in tonal shifts. Recently, methods based on Information Theory [13] and Graph Representations [15] have also emerged.

However, the majority of key modulation algorithms are based on windowed detection, which often results in imprecise localization of modulation points. Reducing the window size may improve the identification of modulation boundaries, but it also weakens overall key detection performance and blurs the distinction between modulation and tonicization.

In this paper, we propose a modulation detection algorithm capable of accurately and efficiently identifying modulation boundaries, without the need to explicitly determine whether modulation is present in a given musical segment. Our method is also readily extendable to frequency-domain analysis and can identify occurrences of any key tonicization. The structure of this paper is as follows: Section 2 outlines related work and the necessary theoretical background. Section 3 details our proposed methodology. Section 4 presents experimental validation based on our approach. Finally, Section 5 concludes the paper and suggests directions for future research.

II. RELATED WORK

Our approach builds upon Chew's Spiral Array theory [7, 8], which has demonstrated considerable potential in the analysis of tonal music. The concept of tonal tension, derived from this model, has proven effective in simulating auditory perceptual motivation [16]. Herremans et al. [17, 18] have explored the feasibility of music generation based on this theory, and more recently, have applied it to multimodal tasks such as video soundtrack generation [19].

A. Basic Elements in Spiral Array theory

In the Spiral Array model, pitch, chord, and key coordinates are computed based on the circle of fifths, forming a three-dimensional helical structure. Specifically, these coordinates are calculated using the following formulas:

Pitch coordinates:

$$P(k) = \begin{bmatrix} x_k \\ y_k \\ z_k \end{bmatrix} = \begin{bmatrix} r\sin(k\pi/2) \\ rcos(k\pi/2) \\ kh \end{bmatrix} \tag{1}$$

Chord coordinates:

$$\begin{aligned} C_M(k) &= w_1 \cdot P(k) + w_2 \cdot P(k+1) + w_3 \cdot P(k+4) \\ C_m(k) &= u_1 \cdot P(k) + u_2 \cdot P(k+1) + u_3 \cdot P(k-3) \end{aligned} \tag{2}$$

Key coordinates:

$$\begin{aligned} T_M(k) &= \omega_1 \cdot C_M(k) + \omega_2 \cdot C_M(k+1) + \omega_3 \cdot C_M(k-1) \\ T_m(k) &= \upsilon_1 \cdot C_m(k) \\ &\quad + \upsilon_2 \cdot [\alpha \cdot C_M(k+1) + (1-\alpha) \cdot C_m(k+1)] \\ &\quad + \upsilon_3 \cdot [\beta \cdot C_m(k-1) + (1-\beta) \cdot C_M(k-1)] \end{aligned} \tag{3}$$

In the formulas, r represents the radius of Spiral Array, while h represents the altitude intercept between adjacent pitches. The $\boldsymbol{w}$, $\boldsymbol{u}$, $\boldsymbol{\omega}$, and $\boldsymbol{v}$ are weight parameters and the sum of all dimensions is 1. By incorporating α and β weights, the model effectively integrates natural minor, melodic minor and harmonic minor scale types into a unified and representative framework. Additionally, parameter k is the index of a note in circle of fifths.

B. Previous Key Detection Work Based on Spiral Array

Chew proposed the Centre of Effect Generator (CEG) Algorithm [8] based on the Spiral Array for key detection, and extended the CEG Algorithm to develop the Boundary Search Algorithm (BSA) [8] to detect key modulation boundaries, enabling the identification of key changes in modulating pieces. Chew's methods have been implemented in the Midi Miner [6] software scripts.

Specifically, the BSA method involves the following: using a dual sliding window to detect modulations. If the *center of effect* (*ce*) [8] distance between pitch sets in the two adjacent windows exceeds a certain threshold, the midpoint between the two windows is marked as a boundary. These boundaries divide the piece into different key segments. However, we find that this method has the following limitations: The method requires prior knowledge of whether the piece modulates to determine whether to apply BSA; Misjudgement between related keys such as C major, a minor, and G major; The method is overly sensitive to tonicizations, failing to distinguish between tonicizations and modulations; Difficulty to balance the window size and detection accuracy. Therefore, this work modifies the elements of Spiral Array and uses hierarchical clustering for effective tonality detection for the first time.

III. Method

A. Determination of Pitch Coordinates in Spiral Array

In our approach, every pitch must be represented consistently. For example, the bVI degree in C major should always appear as either 'A^b' or '$G^\#$', but not both. Subsequently, the component pitches in the Spiral Array and their corresponding index k are as the following sequence: [G: -11, D: -10, A: -9, E: -8, B: -7, G^b: -6, D^b: -5, A^b: -4, E^b: -3, B^b: -2, F: -1, C: 0, G: 1, D: 2, A: 3, E: 4, B: 5, G^b: 6, D^b: 7, A^b: 8, E^b: 9, B^b: 10, F: 11]. Accordingly, given a monotonic or polyphonic progression, the following steps are carried out to determine the index k for all pitches:

Step 1. For each pitch within a time step, iterate through all points and obtain the candidate pitch point set with minimum $\max_{1\le i<j\le N}\left\|P_i(k)-P_j(k)\right\|$, where N is the number of pitch classes.

Step 2. While obtaining multiple candidate sets within the progression, apply beam search algorithm [3] to calculate the optimal point sets distribution in Spiral Array, which satisfies that the total Euclidean distance between *ce* of point sets of adjacent steps is minimized. The *ce* and distance of point sets are computed through equations (4) and (5), respectively. Here, C denotes one chord in a time step.

$$ce_C = \sum_{i=1}^{N} P_i(k) \times \frac{1}{N} \tag{4}$$

$$Distance(C_1 \to C_2) = \left\| ce_{C_1} - ce_{C_2} \right\| \tag{5}$$

The index k of pitch coordinates along the optimal path are considered the final result. Then, all the pitches from a piece can be uniquely defined in Spiral Array. We have implemented a visualization of this model at http://music.designerly.top/spiral_visualization/.

B. Key Modulation Monitoring

Once the positions of all points have been determined, we proceed to compute the overall key of the segment and any tonal changes between segments. Using equations (6) and (7), the key label of a given segment can be derived as follows:

$$ce_{key} = \sum_{j=1}^{M} P_j(k) \times \frac{1}{M} \tag{6}$$

$$f(ce_{key}) = arg\min_{k\in K} \left\| T(k) - ce_{key} \right\| \tag{7}$$

Here, M refers to the total number of pitches in the segment for which we want to calculate tonality, and K is the total indices of key points in the Spiral Array we are modelling.

As tonicization represents a functional, brief departure from the tonal centre, followed by a quick return, any non-diatonic chord can be broadly categorized as tonicization. Modulation, on the other hand, involves a prolonged shift where the new tonal centre is firmly established. Based on these, we propose the boundary search clustering method of key modulation monitoring. Among popular clustering algorithms, such as k-means clustering [2] and hierarchical clustering [4, 5], we chose the latter. In simple terms, the input set of points undergoes the following steps:

Step 1. Arrange the three-dimensional coordinate array of point or point set in time offsets order, and each point or point set is treated as an object b.

Step 2. Calculate equation (8), where L is the length of object array. Then, b_{index} and $b_{index+1}$ could merge into the new object (and their including point sets concatenate).

$$index = arg\min_{1\le i<L} f(b_i,\ b_{i+1}) \tag{8}$$

$$f(b_1,\ b_2) = \begin{cases} \min\limits_{1\le i\le O_1, 1\le j\le O_2} \left\| p_i - p_j \right\|, & single \\ \left\| \sum_{i=1}^{O_1} p_i \times \frac{1}{O_1} - \sum_{j=1}^{O_2} p_j \times \frac{1}{O_2} \right\|, & centroid \end{cases} \tag{9}$$

Here, O_1, O_2 refers to the number of 3D points contained in b_1 and b_2, respectively, and p is the point coordinate. The *centroid* and *single* denote different function.

Step 3. Iterate Step 2 until the entire array becomes one object.

Step 4. Reverse traverse the object array from Step 3 of each iteration to obtain a hierarchical tree. Then, traverse branches backwards from the root of the tree. The loop terminates depending on two predefined threshold parameters that can be adjusted as needed:

1. *off-num*: The acceptable number of tonicization instances. If the number of instances equals or exceeds this parameter, the loop terminates.
2. *off-len*: The length threshold for determining tonicization instance. If the sub-branches number, i.e., the object array's length is below this threshold, the section is treated as a tonicization instance.

Step 5. The last cluster branches are treated as the final result and each object as a segment of the music, and then the key label (using Chew's CEG algorithm or music21's method) is calculated within the scope of each object.

IV. Experiment and Analysis

In modern music, regular songs typically do not modulate more than three times. Based on this, we have provisionally established the following criteria:

- Use the *centroid* function instead of the *single* function in equation (9);
- Set *off-num* to 1;
- Set *off-len* to 3;

A. Evaluation from Non-modulated Examples

We choose POP909 [1] as the evaluation set for popular music and initially selected 10 random songs without key modulations for analysis, specifically: 007, 107, 207, 307, ..., 907. We compare results from four methods: music21, Midi Miner, CEG in key modulation monitoring (KMM), and music21 in KMM, as shown in Table I. Since we excluded enharmonic keys, $c^{\#}$ minor and d^{b} minor were treated as equivalent. Except for 807, where our results slightly differed, all other cases were entirely consistent.

TABLE I. Key Detection Results

File Index	Methods			
	Music21	*Midi Miner*	*CEG in KMM*	*Music21 in KMM*
007	b minor	b minor	b minor	b minor
107	$c^{\#}$ minor	$c^{\#}$ minor	d^{b} minor	$c^{\#}$ minor
207	B major	B major	B major	B major
307	$c^{\#}$ minor	$c^{\#}$ minor	d^{b} minor	$c^{\#}$ minor
407	F major	F major	F major	F major
507	B^{b} major	B^{b} major	B^{b} major	B^{b} major
607	E major	E major	E major	E major
707	B major	B major	B major	B major
807	A^{b} major	A^{b} major	A^{b} - E^{b}	A^{b} major
907	E^{b} major	E^{b} major	E^{b} major	E^{b} major

Upon analysing 807, in fact, A^{b} major and E^{b} major are in a dominant fifth relationship, which caused ambiguity in detection. We draw an analogy to C major scale and G major scale, where the distinction lies between the F pitch and G^{b} pitch. Then, we calculated the *ce* for two pitch sets: [C, D, E, G, A, B] and [C, D, E, F, G^{b}, G, A, B], as shown in Fig. 1, where the pitches show central symmetry in the spiral space. The two sets both identify G major key point as the best candidate. This leads us to hypothesize that if F pitch and G^{b} pitch are equally present in a song—or even if F pitch slightly outnumbers G^{b} pitch—the key result will likely be identified as G major.

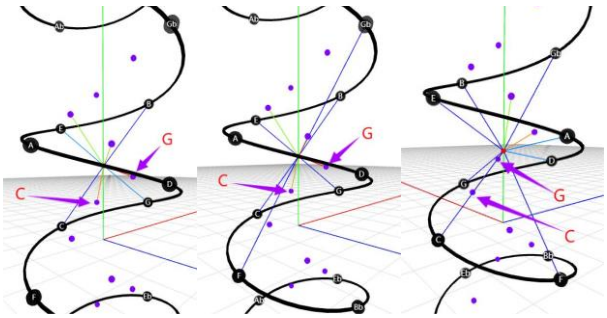

Fig. 1. Three views of pitch sets, their *ce* coordinates, and identified key coordinates. Left: *ce* of [C, D, E, G, A, B]; Middle and right: *ce* of [C, D, E, F, G^{b}, G, A, B]. Purple points: key points. Lines in different colour from *ce* mean distances rank, red < orange < yellow < green < blue. The content in figure could be obtained at visualization website of our model.

Subsequently, we isolate the latter part of Song 807, which has been identified as being in E^{b} major key, and analyze it separately using Midi Miner. The resulting key was also determined to be E^{b} major:

- *key index*: -3 (E^{b})
  ce position: [-0.1718, 0.2844, 0.4468],
  key position: [0.2078, 0.3661, 0.4472],
  distance: 0.3884
- *key index*: -4 (A^{b})
  ce position: [0.2844, 0.1718, 0.7751],
  key position: [0.2078, 0.3661, 0.4472],
  distance: 0.3888

This issue could potentially be addressed by fine-tuning the fundamental parameters of the Spiral Array (r, h, w, u, ω, υ, α and β), or by switching to music21 in KMM function.

We select the 707th song as the representative to demonstrate the Hierarchical clustering process, as shown in Fig. 2. The image highlights two representative features:

- Block similarity (blue boxes): This indicates large-scale repetitive content within the song, reflected in identical or nearly identical tree structures;

- Overall upward trend (red arrows): This is due to the merging of neighboring objects with identical distances, where earlier pitch sets in absolute time step are merged first.

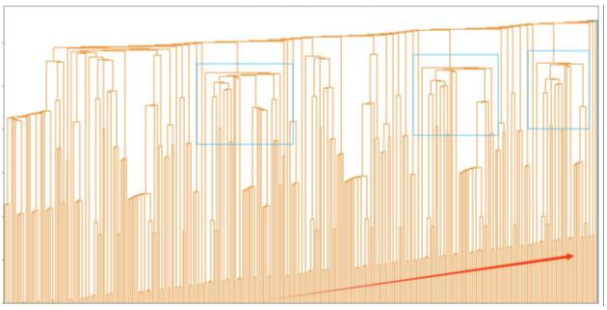

Fig. 2. Hierarchical cluster visualization on number 707 in POP909. X axis shows time step in the piece, while Y axis roughly performance order of objects' mergence.

B. Evaluation from Modulated or Complex Examples

We also select several modulating songs from the POP909 dataset, most of which featured a single modulation. Thus, we choose the 10th song as a representative example, with results visualized using two distinct colours in Fig. 3. For this sample, the key detection results from the four methods are shown in Table II. Music21 in KMM method' result is consistent with the correct answer given by professional musician. Notably, e minor key and G major key represent relative major-minor key pairs.

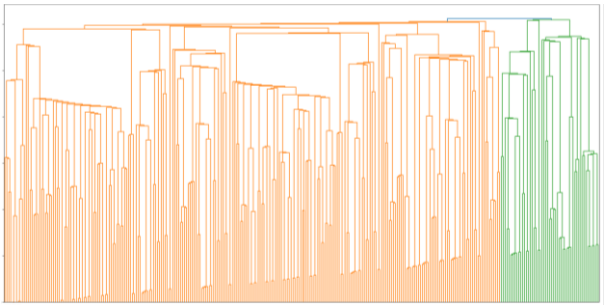

Fig. 3. Hierarchical cluster visualization on number 10 in POP909. The different colours do not denote different keys but rather represent distinct segments identified by the KMM process.

TABLE II. KEY DETECTION RESULTS

Methods	Results
Music21	G major
Midi Miner	G major
CEG in KMM	e minor - A^b major
Music21 in KMM	G major - A^b major

At present, there is no music key modulation dataset with accurate annotation and convenient data extraction to meet the requirements of our verification method. To reduce the visual complexity of the hierarchical cluster tree, and test the model's robustness, we further observe three stylistically distinctive music segments collected from the market, which can be obtained by adding 'pop2.mid', 'bach.mid' or 'jazz.mid' after our visualization page. All the samples are treated into block chord progression form.

Pop2.mid is a more stimulating and complex pop music segment composed by an invited musician. This segment includes more off-tune notes, characterized by frequent modulations and tonicization. The key detection results from CEG in KMM and music21 in KMM are the same: 1–26 in D major, 27–38 in g^b minor, and 39–67 in A major. The hierarchical tree reveals clear segmentation into three sections, with distinct key labels clearly delineated, and the initial continuous tonicizations were captured in isolated branches, shown in Fig. 4.

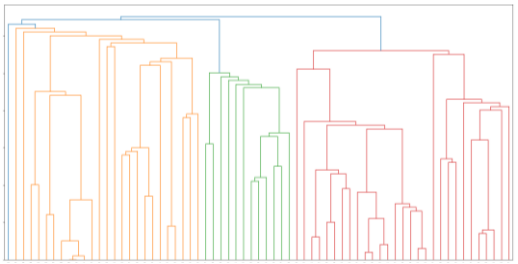

Fig. 4. Hierarchical cluster visualization of pop2.mid.

Bach.mid is a segment of Bach's classical four-part harmony that typical of classical music. Key detection identified the following results for both CEG in KMM and music21 in KMM: 1–23 in d minor and 24–56 in d minor, shown in Fig. 5. But the hierarchical tree suggests further sub-segmentation possibilities. By increasing the *off-num* parameter to 4, the object array is divided at chord indices 9, 23, 42 and 49. The improved segmentation results are as follows: 1–8 in F major, 9–23 in d minor, 24–42 in d minor, 43–49 in C major, and 50–56 in F major, shown in Fig. 6. The truth keys are: 1-8: F major, 9-14: d minor, 15-22: F major, 23-28: d minor, 29-36: F major, 37-42: d minor, 43-50: C major, 51-56: F major.

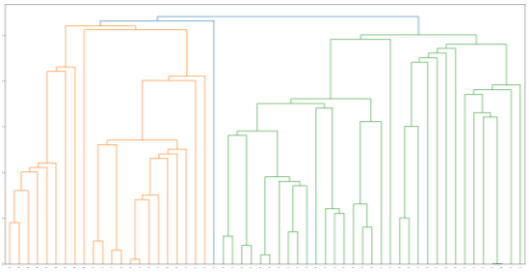

Fig. 5. Hierarchical cluster visualization of bach.mid with *off-num* = 1.

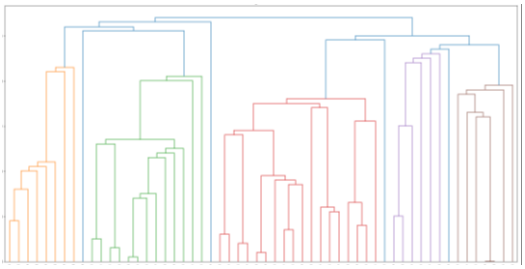

Fig. 6. Hierarchical cluster visualization of bach.mid with *off-num* = 4.

Jazz.mid is a highly modulated, complex jazz chord segment from Bud Powell, representing bebop jazz with frequent key tonicizations and intricate chords. CEG in KMM detect 1–32 in e^b minor and 33–59 in E^b major, while music21 in KMM identify the entire segment as E^b major. The hierarchical tree structure and section divisions are shown in Fig. 7. In fact, e^b minor and E^b major are parallel keys, and music21 in KMM returns the correct result.

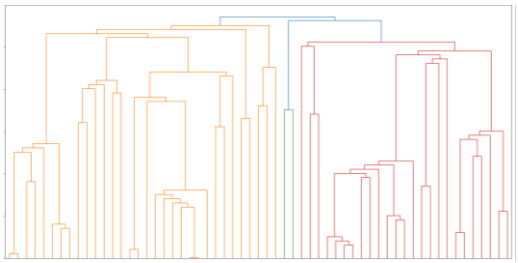

Fig. 7. Hierarchical cluster visualization of jazz.mid.

V. Conclusions

This paper presents a novel approach for key and modulation detection, which is robust to tonicization, accurately segments modulation boundaries, and effectively distinguishes between closely related keys. The method performs well across various musical styles and holds potential for future extension to frequency-domain detection in waveform audio files. However, the current approach is not yet capable of handling intricate tonalities, such as enharmonic keys and modal tonalities (e.g., Lydian, Dorian). We hope that this method can contribute to the construction of music datasets, support music generation tasks, and serve as a useful tool in music composition assistance.

References

[1] Wang, Z., Chen, K., Jiang, J., Zhang, Y., Xu, M., Dai, S., ... & Xia, G. (2020). Pop909: A pop-song dataset for music arrangement generation. In Proceedings of the 21st International Society for Music Information Retrieval Conference (ISMIR).

[2] MacQueen, J. (1967, June). Some methods for classification and analysis of multivariate observations. In Proceedings of the fifth Berkeley symposium on mathematical statistics and probability (Vol. 1, No. 14, pp. 281-297).

[3] Clara Meister, Tim Vieira, Ryan Cotterell. 2020. Best-first beam search. Transactions of the Association for Computational Linguistics, 8, 795-809. https://doi.org/10.1162/tacl_a_00346

[4] Johnson, S. C. (1967). Hierarchical clustering schemes. Psychometrika, 32(3), 241-254.

[5] Lance, G. N., & Williams, W. T. (1967). A general theory of classificatory sorting strategies: 1. Hierarchical systems. The computer journal, 9(4), 373-380.

[6] Guo, R., Herremans, D., & Magnusson, T. (2019). Midi Miner--A Python library for tonal tension and track classification. Late Breaking/Demo at the 20th International Society for Music Information Retrieval (ISMIR).

[7] Chew, E. (2000). Towards a mathematical model of tonality (Doctoral dissertation, Massachusetts Institute of Technology).

[8] Chew, E. (2014). Mathematical and Computational Modeling of Tonality. Springer.

[9] Krumhansl, C. L., & Schmuckler, M. A. (1986). Key-finding in music: An algorithm based on pattern matching to tonal hierarchies. In 19th Annual Meeting of the Society of Mathematical Psychology, Cambridge, MA.

[10] Richard Cohn. Neo-riemannian operations, parsimonious trichords, and their" tonnetz" representations. Journal of Music Theory 41.1 (1997): 1-66.

[11] Cuthbert, M. S., & Ariza, C. (2010). music21: A toolkit for computer-aided musicology and symbolic music data. In International Society for Music Information Retrieval Conference (ISMIR) (pp. 637-642).

[12] Tsung-Ping Chen, and Li Su. "Functional Harmony Recognition of Symbolic Music Data with Multi-task Recurrent Neural Networks." ISMIR. 2018.

[13] Ventura, M.D. (2022). Automatic Recognition of Key Modulations in Symbolic Musical Pieces Using Information Theory. In: Arai, K. (eds) Intelligent Systems and Applications. IntelliSys 2021. Lecture Notes in Networks and Systems, vol 294. Springer, Cham. https://doi.org/10.1007/978-3-030-82193-7_56

[14] Yuexuan Kong, Gabriel Meseguer-Brocal, Vincent Lostanlen, Mathieu Lagrange, Romain Hennequin. "S-KEY: Self-supervised Learning of Major and Minor Keys from Audio." arXiv preprint arXiv:2501.12907 (2025).

[15] Hernandez-Olivan, Carlos, Sonia Rubio Llamas, and Jose R. Beltran. "Symbolic music structure analysis with graph representations and changepoint detection methods." arXiv preprint arXiv:2303.13881 (2023).

[16] Herremans, D., & Chew, E. (2016). Tension ribbons: Quantifying and visualising tonal tension. In Second International Conference on Technologies for Music Notation and Representation (TENOR) (pp. 8-18).

[17] Guo, R., Simpson, I., Magnusson, T., Kiefer, C., & Herremans, D. (2020). A variational autoencoder for music generation controlled by tonal tension. Joint Conference on AI Music Creativity (CSMC-MuMe).

[18] Herremans, D., & Chew, E. (2017). MorpheuS: generating structured music with constrained patterns and tension. IEEE Transactions on Affective Computing, 10(4), 510-523.

[19] Kang, J., Poria, S., & Herremans, D. (2024). Video2music: Suitable music generation from videos using an affective multimodal transformer model. Expert Systems with Applications, 249, 123640.